

PRACTICAL : 4

1. Find the average number of friends by age using the data given.

Code:

```
from pyspark.sql import SparkSession
from pyspark.sql import Row
from pyspark.sql import functions as Func

spark = SparkSession.builder.appName("FriendByAge").getOrCreate()

lines=spark.read.option("header","true").option("inferSchema","true").csv("/Volumes/workspace/default/avgfriends_37/avgfriends.csv")

friendsByAge=lines.select("age","friends")

friendsByAge.groupBy("age").avg("friends").sort("age").show()

friendsByAge.groupBy("age").agg(Func.round(Func.avg("friends"),2)).sort("age").show()

friendsByAge.groupBy("age").agg(Func.round(Func.avg("friends"),2)
    .alias("friends_avg")).sort("age").show()
```

Output:

```
> See performance \(3\)
> friendsByAge: pyspark.sql.connect.dataframe.DataFrame = [age: integer, fri
> lines: pyspark.sql.connect.dataframe.DataFrame = [sr: integer, name: string
```

age	avg(friends)
18	343.375
19	213.27272727272728
20	165.0
21	350.875
22	206.42857142857142
23	246.3
24	233.8
25	197.45454545454547
26	242.05882352941177
27	228.125
28	209.1
29	215.91666666666666
30	235.8181818181818
31	267.25
32	207.9090909090909
33	325.3333333333333
34	245.5
35	211.625

2. use the dataset given and write the code to find the minimum temperature by the location (each whether station)and understand it and modify it to find maximum temperature by the location.

Code:

```
from pyspark.sql.functions import min, max, col
path = "/Volumes/bda/default/temp/temp (1).csv"
df = spark.read.csv(path, inferSchema=True)
df = df.select(col("_c0").alias("StationID"), col("_c2").alias("TempType"),
col("_c3").alias("TempValue"))

min_results = df.filter(df["TempType"] ==
"TMIN").groupBy("StationID").agg(min("TempValue").alias("MinTemp")).orderBy("StationID").collect()
max_results = df.filter(df["TempType"] ==
"TMAX").groupBy("StationID").agg(max("TempValue").alias("MaxTemp")).orderBy("StationID").collect()

print("Minimum Temperatures by Station (Celsius):")
for row in min_results: print(f'Station: {row["StationID"]} Min Temp: {row["MinTemp"]}/10:.2f}°C')
print("\nMaximum Temperatures by Station (Celsius):")
for row in max_results: print(f'Station: {row["StationID"]} Max Temp: {row["MaxTemp"]}/10:.2f}°C')
```

Output:

```
> df: pyspark.sql.connect.dataframe.DataFrame = [StationID: string, TempType: string ... 1 more field]

Minimum Temperatures by Station (Celsius):
Station: EZE00100082 Min Temp: -13.50°C
Station: ITE00100554 Min Temp: -14.80°C

Maximum Temperatures by Station (Celsius):
Station: EZE00100082 Max Temp: 32.30°C
Station: ITE00100554 Max Temp: 32.30°C
```

3. Analyze the provided customer spending dataset to calculate the total expenditure for each person, utilizing an RDD in the notebook and sorting the final output by the total amount spent. (customerorders.csv)

Code:

```
from pyspark.sql.functions import sum, round
path = "/Volumes/bda/default/customer/customerorders (1).csv"
df = spark.read.csv(path, inferSchema=True).toDF("CustomerID", "TransactionID",
"Amount")
result = df.groupBy("CustomerID").agg(round(sum("Amount"),
2).alias("TotalSpent")).orderBy("TotalSpent").collect()
for row in result:
    print(f'Customer ID: {row['CustomerID']} Total Spent: {row['TotalSpent']:.2f}')
```

Output:

```
> df: pyspark.sql.connect.dataframe.DataFrame = [CustomerID: integer, TransactionID: integer ... 1 more field]
Customer ID: 45 Total Spent: 3309.38
Customer ID: 79 Total Spent: 3790.57
Customer ID: 96 Total Spent: 3924.23
Customer ID: 23 Total Spent: 4042.65
Customer ID: 99 Total Spent: 4172.29
Customer ID: 75 Total Spent: 4178.50
Customer ID: 36 Total Spent: 4278.05
Customer ID: 98 Total Spent: 4297.26
Customer ID: 47 Total Spent: 4316.30
Customer ID: 77 Total Spent: 4327.73
Customer ID: 13 Total Spent: 4367.62
Customer ID: 48 Total Spent: 4384.33
Customer ID: 49 Total Spent: 4394.60
Customer ID: 94 Total Spent: 4475.57
Customer ID: 67 Total Spent: 4505.79
Customer ID: 50 Total Spent: 4517.27
Customer ID: 78 Total Spent: 4524.51
Customer ID: 5 Total Spent: 4561.07
Customer ID: 57 Total Spent: 4628.40
Customer ID: 83 Total Spent: 4635.80
Customer ID: 91 Total Spent: 4642.26
```

4. Calculate the frequency of each word within a provided text dataset. Utilize regular expressions to normalize the text before performing the word count and sorting the final results. (wordcount data.txt)

Code:

```
from pyspark.sql.functions import explode, split, lower, regexp_replace, col
path = "/Volumes/bda/default/wordcount/wordcount data (1).txt"
df = spark.read.text(path)
words = df.select(explode(split(col("value"), "\\s+")).alias("word"))
clean = words.select(lower(regexp_replace(col("word"), "[^a-zA-Z0-9]",
""))).alias("word").filter("word != """)
result = clean.groupBy("word").count().orderBy(col("count").asc()).collect()
for row in result: print(f'{row["word"]}: {row["count"]}')
```

Output:

```
> clean: pyspark.sql.connect.dataframe.DataFrame = [word: string]
> df: pyspark.sql.connect.dataframe.DataFrame = [value: string]
> words: pyspark.sql.connect.dataframe.DataFrame = [word: string]

partly: 1
updating: 1
value: 1
complex: 1
greater: 1
statistical: 1
power: 1
columns: 1
false: 1
lack: 1
visualization: 1
quality: 1
expertise: 1
from: 1
are: 1
software: 1
complexity: 1
interpretation: 1
storage: 1
search: 1
transfer: 1
```